데이터베이스(SQL)

4장. SQL 기초 - 데이터 조회

학습 목표

1) SQL의 역할을 설명할 수 있다.
2) SQL의 작성 순서와 실행 순서 차이를 설명할 수 있다.
3) SELECT, WHERE로 원하는 데이터를 조회할 수 있다.
4) 비교 연산자와 논리 연산자를 사용할 수 있다.
5) IN, BETWEEN, LIKE, ILIKE로 조건을 지정할 수 있다.
6) NULL을 올바르게 처리할 수 있다.
7) DISTINCT, ORDER BY, LIMIT, OFFSET을 사용할 수 있다.
8) 가독성 있는 SQL을 작성할 수 있다.

공식 문서: SELECT 문

전체 흐름


1)
SQL 분류
2)
작성 순서와
처리 순서
3)
SELECT
4)
WHERE
5)
논리 연산자
6)
IN · BETWEEN ·
LIKE · ILIKE
7)
NULL
8)
DISTINCT
9)
ORDER BY
10)
LIMIT · OFFSET

01 SQL과 처리 순서

SQL과 명령어 분류

SQL - 관계형 데이터베이스(RDBMS)에 저장된 데이터를 관리하고 처리하기 위해 사용하는 표준 컴퓨터 언어

SQL 명령어 분류

분류	정식 명칭	역할	대표 명령어
DQL	Data Query Language	조회	SELECT
DML	Data Manipulation Language	조작	INSERT, UPDATE, DELETE
DDL	Data Definition Language	정의	CREATE, ALTER, DROP
DCL	Data Control Language	권한	GRANT, REVOKE
TCL	Transaction Control Language	트랜잭션	COMMIT, ROLLBACK

작성 순서와 논리적 처리 순서

작성 순서



처리 순서



SELECT 문의 처리 과정

SELECT 문의 기본 형식

SELECT 문은 관계 데이터베이스에서 데이터를 검색하는 기본 문장이다.

```
SELECT 컬럼명 목록
FROM 테이블명
WHERE 조건식;
```

SELECT, FROM, WHERE처럼 SQL에서 정해진 의미를 가진 단어를 **키워드**라고 한다.

절	적는 내용	의미
SELECT	검색하고 싶은 컬럼: *, 컬럼, 산술 연산식, 함수(COUNT, SUM, MIN, MAX 등)	열(컬럼) 선택
FROM	검색에 필요한 테이블 이름	데이터를 가져올 곳
WHERE	검색 조건. 생략할 수 있다	행(레코드) 검색 조건

공식 문서: SELECT

SELECT 문의 작동 원리

```
SELECT salary
FROM employees
WHERE ename = 'EVE';
```

처리 순서

FROM → WHERE → SELECT

결과

salary
30000

1) FROM employees : 테이블 전체를 가져온다

ssn	ename	age	salary	dno
1111	ABE	45	30000	d1
2222	**EVE**	**28**	**30000**	**d2**
3333	JIM	37	35000	d2
4444	BOB	35	40000	d3
5555	PAUL	25	17000	d3
6666	DICK	35	32000	d3

2) WHERE ename = 'EVE' 행을 고른다

3) SELECT salary : 열을 고른다

컬럼	의미
ssn	사원 번호
ename	사원 이름
age	나이
salary	급여
dno	부서 번호

이름이 'EVE'인 사원의 salary를 검색한다. WHERE는 행을, SELECT는 열을 고른다

공식 문서: SELECT

GROUP BY 절 - 그룹화

- GROUP BY는 **같은 값을 가진 행끼리 하나의 그룹으로 묶는다.**

- 묶은 그룹마다 COUNT(*)로 행 개수를 셀 수 있다.

```
SELECT dno, COUNT(*) AS 인원
FROM employees
WHERE age > 20
GROUP BY dno;
```

결과

dno	인원
d1	1
d2	2
d3	3

공식 문서: 테이블 표현식(WHERE · GROUP BY · HAVING) / 집계 함수

GROUP BY 처리 과정

1) FROM · WHERE age > 20

ssn	ename	age	dno
1111	ABE	45	d1
2222	EVE	28	d2
3333	JIM	37	d2
4444	BOB	35	d3
5555	PAUL	25	d3
6666	DICK	35	d3

age > 20 : 6행 모두 통과

2) GROUP BY dno

d1	1111	ABE	45
d2	2222	EVE	28
	3333	JIM	37
d3	4444	BOB	35
	5555	PAUL	25
	6666	DICK	35

묶기만 한다 : 컬럼은 그대로 있다

그룹마다 개수

3) SELECT : 그룹마다 개수 세기

dno	인원
d1	1
d2	2
d3	3

그룹 하나가 결과 한 행이 된다

SELECT에 적은 dno와 인원만 남는다

처리 순서: FROM → WHERE → GROUP BY → SELECT

같은 dno 값을 가진 행을 하나로 묶고, 묶음마다 COUNT(*)로 행 수를 센다

공식 문서: 테이블 표현식(WHERE · GROUP BY · HAVING)

HAVING 절 - 그룹에 검색 조건 지정

HAVING 절을 이용하여 GROUP BY 절이 만든 결과 중 **조건에 맞는 그룹만** 표시할 수 있다.

```
SELECT dno, COUNT(*) AS 인원
FROM employees
WHERE age > 20
GROUP BY dno
HAVING COUNT(*) > 1;
```

구분	WHERE	HAVING
거르는 대상	행(레코드)	그룹
처리 시점	GROUP BY 전	GROUP BY 후

공식 문서: 테이블 표현식(WHERE · GROUP BY · HAVING)

HAVING 처리 과정

GROUP BY 결과

dno	인원
d1	1
d2	2
d3	3

1 > 1 : 거짓

2 > 1 : 참

3 > 1 : 참

HAVING COUNT(*) > 1

HAVING 결과

dno	인원
d2	**2**
d3	**3**

조건이 참인 그룹만 남는다

처리 순서: FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY

HAVING은 묶인 그룹마다 조건을 확인한다. d1은 인원이 1명이라 빠진다

공식 문서: 테이블 표현식(WHERE · GROUP BY · HAVING)

ORDER BY와 LIMIT - 정렬과 개수 제한

```
SELECT ename, age
FROM employees
ORDER BY age DESC
LIMIT 2;
```

ORDER BY : 결과를 정렬한다. ASC(기본)는 작은 값부터, DESC는 큰 값부터.

LIMIT n : 정렬된 결과에서 앞의 n행만 남긴다.

1) FROM · SELECT ename, age

ename	age
ABE	45
EVE	28
JIM	37
BOB	35
PAUL	25
DICK	35

2) ORDER BY age DESC

ename	age
ABE	**45**
JIM	**37**
BOB	35
DICK	35
EVE	28
PAUL	25

나이가 많은 순서로 정렬

3) LIMIT 2

ename	age
ABE	**45**
JIM	**37**

앞에서 2행만 남긴다

공식 문서: 행 정렬 / LIMIT와 OFFSET

전체 처리 과정 따라가기

```
SELECT dno, COUNT(*) AS 인원
FROM employees
WHERE age > 20
GROUP BY dno
HAVING COUNT(*) > 1
ORDER BY 인원 DESC
LIMIT 1;
```

결과

dno	인원
d3	3

1) **FROM employees** — 6행을 가져온다
2) **WHERE age > 20** — 6행 모두 통과
3) **GROUP BY dno** — d1 · d2 · d3 세 그룹
4) **HAVING COUNT(*) > 1** — d2 · d3 두 그룹만 남는다
5) **SELECT dno, 인원** — 그룹마다 개수를 세고 별칭을 붙인다
6) **ORDER BY 인원 DESC** — d3(3명) → d2(2명) 순서
7) **LIMIT 1** — 첫 행 d3만 남긴다

공식 문서: SELECT / 행 정렬 / LIMIT와 OFFSET

검색을 위한 SQL 문 요약

처리 순서

FROM → WHERE → GROUP BY → HAVING → SELECT → DISTINCT → ORDER BY → LIMIT

절	처리 순서	하는 일
SELECT	5	조회할 컬럼을 선택한다
DISTINCT	6	중복된 행을 하나만 남긴다
FROM	1	데이터 조회 대상이 되는 테이블을 지정한다
WHERE	2	조건을 지정하여 특정 레코드를 필터링한다
GROUP BY	3	데이터를 그룹화할 기준 컬럼을 지정한다
HAVING	4	그룹화된 결과에 대한 조건을 지정한다
ORDER BY	7	조회 결과를 정렬 기준에 따라 정렬한다
LIMIT	8	결과 레코드 수를 제한한다

공식 문서: SELECT

02 SELECT와 WHERE

SELECT - 데이터 조회

기본 문법

```
SELECT 컬럼명
FROM 테이블명;
```

전체 컬럼 조회

```
SELECT *
FROM customers;
```

특정 컬럼만 조회

```
SELECT
    customer_id,
    company_name,
    city
FROM customers;
```

공식 문서: Select 목록

별칭(Alias) 사용

```
SELECT
    customer_id AS 고객번호,
    company_name AS 회사명,
    city AS 도시
FROM customers;
```

AS 뒤에 결과 컬럼에 보일 이름을 적는다.

SELECT * 를 피해야 하는 이유

이유	설명
불필요한 데이터 전송	필요 없는 열까지 읽고 보내서 데이터 양이 커진다
테이블 구조 변경에 취약	컬럼이 추가되면 결과 컬럼 구성이 달라질 수 있다
가독성 저하	어떤 데이터를 쓰는지 알 수 없다

탐색 단계에서는 SELECT *로 확인하고, 실제 분석 쿼리에서는 필요한 컬럼만 명시한다.

WHERE - 조건 필터링

기본 문법

```
SELECT *
FROM customers
WHERE city = 'London';
```

비교 연산자

연산자	의미	예시
=	같다	city = 'London'
<> 또는 !=	같지 않다	country <> 'USA'
>	크다	unit_price > 100
<	작다	unit_price < 50
>=	크거나 같다	quantity >= 10
<=	작거나 같다	quantity <= 100

공식 문서: WHERE 절

WHERE에서 별칭을 쓸 수 없는 이유

SELECT가 WHERE보다 나중에 실행되기 때문에, SELECT에서 만든 별칭(alias)을 WHERE에서 쓸 수 없다.

```
-- 오류 발생: WHERE 실행 시점에는 별칭이 아직 없음
SELECT unit_price * 2 AS 두배가격
FROM products
WHERE 두배가격 > 100;
```

오류: "두배가격" 이름의 칼럼은 없습니다 / : ERROR: column "두배가격" does not exist

별칭 오류 해결

올바른 방법: 원래 식을 그대로 사용

```
-- 올바른 방법: 원래 식을 그대로 사용
SELECT unit_price * 2 AS 두배가격
FROM products
WHERE unit_price * 2 > 100;
```

반면 ORDER BY는 SELECT 이후에 실행되므로 별칭 사용이 가능하다.

비교 연산자 예제

```sql
-- 단가가 50 이상인 제품
SELECT
  product_name,
  unit_price
FROM products
WHERE unit_price >= 50;
```

```sql
-- 미국이 아닌 고객
SELECT
  company_name,
  country
FROM customers
WHERE country <> 'USA';
```

AND, OR, NOT - 논리 연산자

AND - 모든 조건 만족

```
SELECT *
FROM customers
WHERE country = 'UK'
AND city = 'London';
```

OR - 하나라도 만족

```
SELECT *
FROM customers
WHERE city = 'London'
OR city = 'Paris';
```

NOT - 조건 반대

```
SELECT *
FROM customers
WHERE NOT country = 'USA';
```

공식 문서: 논리 연산자

연산자 우선순위

AND가 OR보다 먼저 계산된다. 의도한 대로 동작하게 하려면 괄호를 사용한다.

```
-- 괄호 없음: (country='UK' AND city='London') OR city='Paris'
SELECT * FROM customers
WHERE country = 'UK' AND city = 'London' OR city = 'Paris';

-- 괄호 사용: 의도가 명확함
SELECT * FROM customers
WHERE country = 'UK' AND (city = 'London' OR city = 'Paris');
```

03 IN, BETWEEN, LIKE, ILIKE

IN - 여러 값 비교

OR를 여러 번 쓰는 대신 간결하게 표현한다.

```
-- OR 방식
SELECT * FROM customers
WHERE city = 'London'
OR city = 'Paris'
OR city = 'Berlin';

-- IN 방식 (더 간결)
SELECT * FROM customers
WHERE city IN ('London', 'Paris', 'Berlin');
```

공식 문서: IN 연산자

NOT IN과 NULL

```
SELECT * FROM customers
WHERE country NOT IN ('USA', 'Canada', 'Mexico');
```

NOT IN 목록에 NULL이 섞이면 결과가 한 건도 나오지 않을 수 있다.

BETWEEN - 범위 조회

BETWEEN은 시작값과 종료값을 **모두 포함**한다.

```
BETWEEN 1 AND 10
```

위 표현은 아래와 같다.

```
>= 1 AND <= 10
```

숫자 범위

```
SELECT
product_name,
unit_price
FROM products
WHERE unit_price BETWEEN 20 AND 50;
```

공식 문서: 비교 연산자 (BETWEEN)

BETWEEN 날짜 범위와 시간 포함 주의

날짜 범위 — BETWEEN

```
SELECT
order_id,
order_date
FROM orders
WHERE order_date BETWEEN '1997-01-01' AND '1997-01-31';
```

주의 - 날짜에 시간이 포함된 경우

BETWEEN '1997-01-01' AND '1997-01-31' 처럼 쓰면 종료값이 '1997-01-31 00:00:00' 으로 해석된다.
컬럼이 TIMESTAMP 타입이면 **1월 31일 낮 시간대 데이터가 빠진다.**

시작일 이상, 다음 달 1일 미만으로 쓰면 안전하다.

LIKE - 문자열 패턴 검색

와일드카드

기호	의미
%	0개 이상의 임의 문자
_	정확히 1개의 임의 문자

패턴 예시

패턴	의미	매칭 예
'A%'	A로 시작	Alfreds, Ana
'%s'	s로 끝남	Alfreds, Sales
'%an%'	an 포함	France, Islands
'A__'	A로 시작하는 3글자	ABC, AXY
'_o%'	두 번째 글자가 o	London, Rome

공식 문서: 패턴 매칭

LIKE 예제

```sql
-- 회사명에 'Rest'가 포함된 고객
SELECT company_name
FROM customers
WHERE company_name LIKE '%Rest%';
```

```sql
-- B로 시작하는 도시
SELECT DISTINCT city
FROM customers
WHERE city LIKE 'B%';
```

ILIKE - 대소문자 구분 없는 검색

연산자	대소문자	표준/특징
LIKE	대소문자를 구분한다	SQL 표준 연산자
ILIKE	대소문자를 구분하지 않는다	PostgreSQL 확장

```
-- LIKE: 'london'은 못 찾음
SELECT * FROM customers WHERE city LIKE 'london';
-- 결과: 0건

-- ILIKE: 'London' 찾음
SELECT * FROM customers WHERE city ILIKE 'london';
-- 결과: 6건
```

LIKE 'london' → 0건

ILIKE 'london' → 6건

공식 문서: 패턴 매칭 (LIKE, ILIKE)

ILIKE 활용

사용자 입력 검색처럼 대소문자를 예측할 수 없을 때 ILIKE를 쓴다.

```
-- 검색어가 'REST', 'rest', 'Rest' 무엇이든 찾음
SELECT company_name
FROM customers
WHERE company_name ILIKE '%rest%';
```

04 NULL 처리

NULL이란

NULL은 값이 없음(모름)을 뜻한다. 0이나 빈 문자열('')은 값이 있는 것이므로 NULL과 다르다.

값	의미
0	숫자 영 (값이 있음)
''	길이가 0인 문자열 (값이 있음)
NULL	값이 없음 (모름)

공식 문서: 비교 연산자와 NULL

NULL 비교는 = 로 할 수 없다

```
-- 잘못된 예: 아무 결과도 안 나옴
SELECT * FROM customers WHERE region = NULL;

-- 올바른 예
SELECT * FROM customers WHERE region IS NULL;
SELECT * FROM customers WHERE region IS NOT NULL;
```

region = NULL → 아무 결과도 안 나옴

region IS NULL / IS NOT NULL → 올바른 비교

왜 = 가 안 되나 NULL은 “알 수 없는 값”이다. 알 수 없는 값끼리 같은지 비교하면 결과도 “알 수 없음”이 된다.

NULL 연산 주의

```
SELECT NULL + 100; -- NULL (100이 아님)
SELECT NULL || '텍스트'; -- NULL
SELECT NULL = NULL; -- NULL (TRUE 아님)
```

일반적인 산술·문자열 연결·등호 비교에서 NULL이 포함되면 결과는 NULL이다. 다만 논리 연산 등에는 예외가 있다.

NULL 확인 조건과 표현식

조건·표현식	역할
IS NULL	NULL인지 확인
IS NOT NULL	NULL이 아닌지 확인
COALESCE(a, b)	a가 NULL이면 b, 아니면 a
NULLIF(a, b)	a와 b가 같으면 NULL, 다르면 a

```
-- region이 NULL이면 '미지정'으로 표시
SELECT
    company_name,
    COALESCE(region, '미지정') AS 지역
FROM customers
LIMIT 10;

-- 나누는 값이 0이면 오류 대신 NULL (NULLIF 활용)
SELECT 10 / NULLIF(0, 0);   -- NULL
SELECT 10 / NULLIF(2, 0);   -- 5
```

05 DISTINCT와 정렬

DISTINCT - 중복 제거

DISTINCT는 조회 결과에서 중복된 행을 제외하여 조회한다.

```
-- 고객들이 사는 도시 목록 (중복 없이)
SELECT DISTINCT city
FROM customers
ORDER BY city;
```

여러 컬럼 DISTINCT

```
-- 국가 + 도시 조합이 유일한 것만
SELECT DISTINCT country, city
FROM customers
ORDER BY country, city;
```

country와 city를 **합쳐서** 중복을 판단한다.

공식 문서: DISTINCT 절

ORDER BY - 정렬

기본 문법

```
SELECT
    product_name,
    unit_price
FROM products
ORDER BY unit_price DESC;
```

정렬 방향

키워드	의미	기본값
ASC	오름차순 (작은 것부터)	기본값
DESC	내림차순 (큰 것부터)	명시 필요

ORDER BY가 없으면 결과 순서가 보장되지 않는다. 순서가 중요하면 반드시 ORDER BY를 사용한다

공식 문서: 정렬

여러 컬럼 정렬

```
SELECT
    country,
    city,
    company_name
FROM customers
ORDER BY country ASC, city ASC, company_name ASC;
```

앞 컬럼부터 순서대로 정렬한다.
정렬 방향은 컬럼마다 따로 지정되므로
ORDER BY country, city DESC 는 country는 오름차순, city만 내림차순이다.

NULL 정렬 위치

NULL 정렬

PostgreSQL은 오름차순(ASC)일 때 NULL을 맨 뒤에, 내림차순(DESC)일 때 맨 앞에 둔다.
위치를 직접 정하려면 NULLS FIRST / NULLS LAST를 쓴다.

```
-- NULL을 항상 마지막으로
SELECT company_name, region
FROM customers
ORDER BY region NULLS LAST;

-- NULL을 항상 처음으로
SELECT company_name, region
FROM customers
ORDER BY region NULLS FIRST;
```

06 LIMIT, OFFSET

LIMIT - 개수 제한

```
-- 상위 10개만
SELECT
    product_name,
    unit_price
FROM products
ORDER BY unit_price DESC
LIMIT 10;
```

```
-- 데이터 일부 확인
SELECT * FROM orders LIMIT 5;
```

공식 문서: LIMIT과 OFFSET

OFFSET - 건너뛰기

페이징 처리

```
-- 1페이지 (1~10번째)
SELECT * FROM products ORDER BY product_id LIMIT 10 OFFSET 0;

-- 2페이지 (11~20번째)
SELECT * FROM products ORDER BY product_id LIMIT 10 OFFSET 10;

-- 3페이지 (21~30번째)
SELECT * FROM products ORDER BY product_id LIMIT 10 OFFSET 20;
```

페이지 계산 공식

```
OFFSET = (페이지번호 - 1) × 페이지당개수
```

공식 문서: LIMIT과 OFFSET

핵심 정리

1) SQL은 작성 순서와 논리적 처리 순서가 다르다.
2) SELECT에서 만든 별칭은 WHERE에서 쓸 수 없고, ORDER BY에서는 쓸 수 있다.
3) 실제 분석 쿼리에서는 SELECT * 대신 필요한 컬럼만 명시한다.
4) AND는 OR보다 먼저 계산되므로 괄호로 의도를 명확히 한다.
5) IN은 여러 값 비교를 간결하게 표현한다.
6) BETWEEN은 시작값과 종료값을 모두 포함한다.
7) LIKE는 대소문자를 구분하고 ILIKE는 구분하지 않는다. ILIKE는 PostgreSQL 전용이다.
8) NULL은 0이나 빈 문자열과 다르며 IS NULL로 비교해야 한다.
9) 일반적인 산술·비교에서 NULL은 NULL 결과를 낸다. 논리 연산 등에는 예외가 있다.
10) DISTINCT는 조회 결과의 중복만 제거하며 실제 데이터는 삭제하지 않는다.
11) ORDER BY 없이는 결과 순서가 보장되지 않는다.
12) LIMIT과 OFFSET으로 페이징을 처리한다.

스타일 가이드 참고 SQL Style Guide: https://www.sqlstyle.guide/